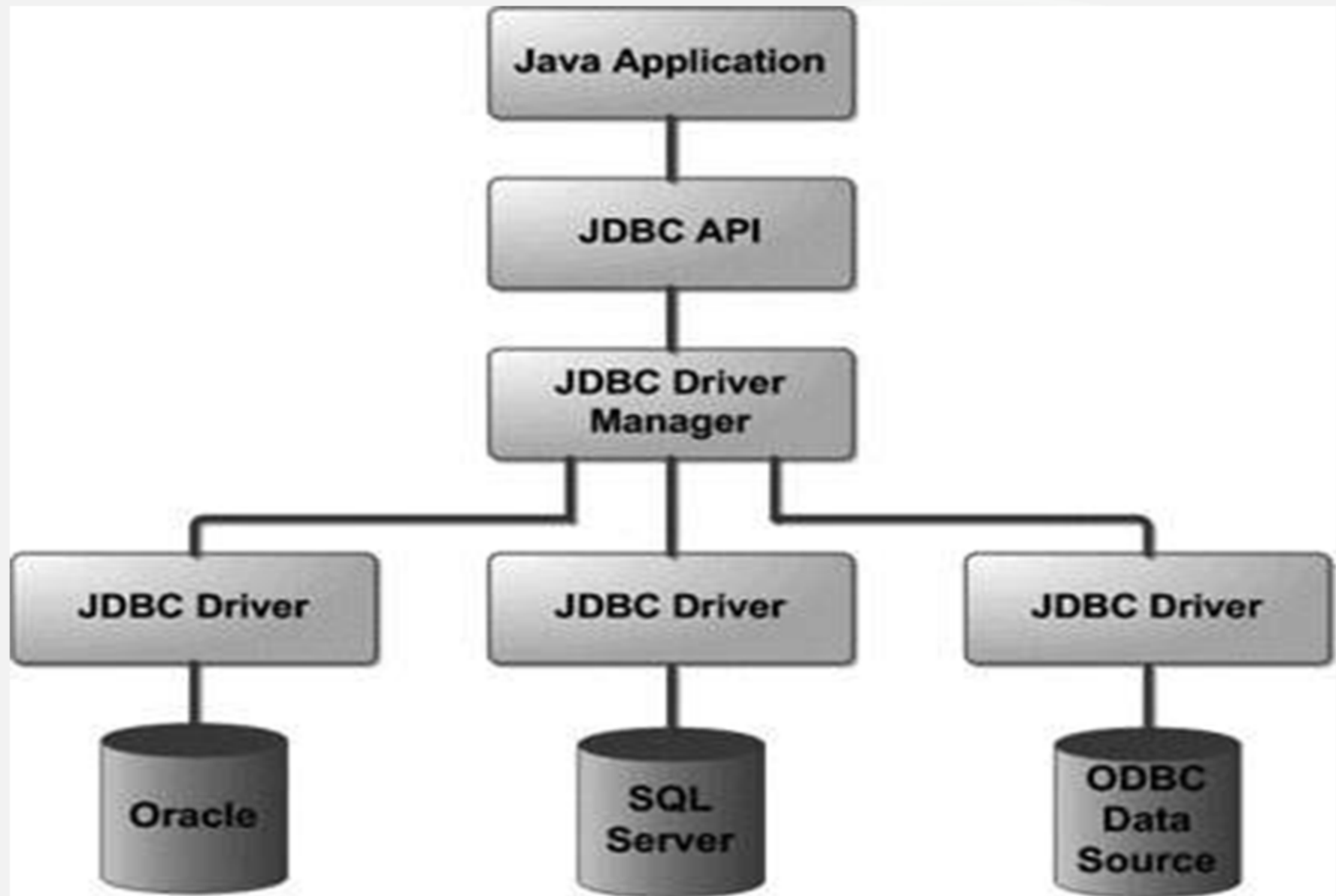




JDBC I

- What is JDBC
- JDBC architecture
- JDBC specification
- JDBC driver types
- Driver selection
- Popular JDBC driver names and database URL
- Registering the driver
- Establishing connections
- Creating statement
- Executing statements
- Example for simple select query
- Example for DML query
- Closing connection

- JDBC stands for Java Database Connectivity, which is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases.
- The JDBC API is a Java API that can access any kind of tabular data, especially data stored in a Relational Database.
- It works with Java on various kinds of platforms like Windows, Linux, MacOS etc.

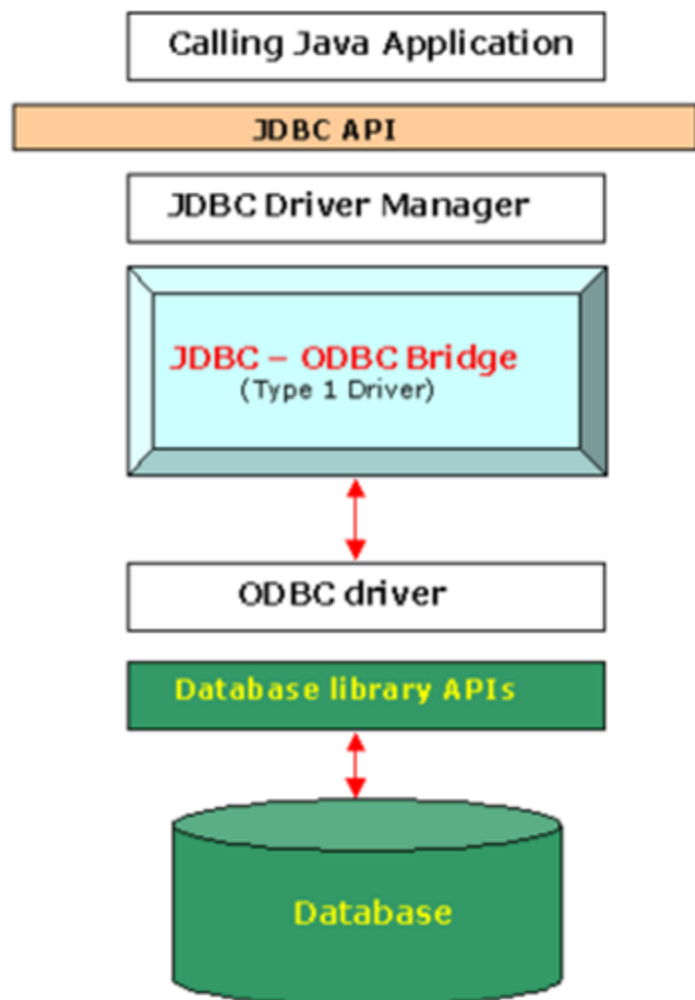


- The JDBC API proposed will support both two-tier and three-tier architecture flows.
- In general the architecture is comprised of two layers,
 - ◆ JDBC API – Java application to Driver Manager Connection
 - ◆ JDBC Driver Manager – JDBC Manager to Driver Connection
- JDBC API provides transparent connectivity using the Driver Manager and database-specific drivers. For this two packages `java.sql.*` and `javax.sql.*` are provided. These two packages exhibit Write Once Run Everywhere capability which contains both Java SE and Java EE support.
- JDBC Driver Manager provides assurance for the usage of the correct database driver to access each data source. It manages the list of database drivers.

- Fundamentally, JDBC is a specification that provides a complete set of interfaces that allows for portable access to an underlying database. Java can be used to write different types of executables, such as Java Applications, Java Servlets, Java ServerPages (JSPs) etc.
- JDBC drivers are the implementations of JDBC specification. JDBC drivers are specific to the particular database and can be downloaded in the form of jar file.
- This jar file consists of set of classes that implement the interfaces in the JDBC specification.
- JDBC help to write Java applications that manage these three programming activities:
 - ◆ Connect to a data source, like a database
 - ◆ Send queries and update statements to the database
 - ◆ Retrieve and process the results received from the database

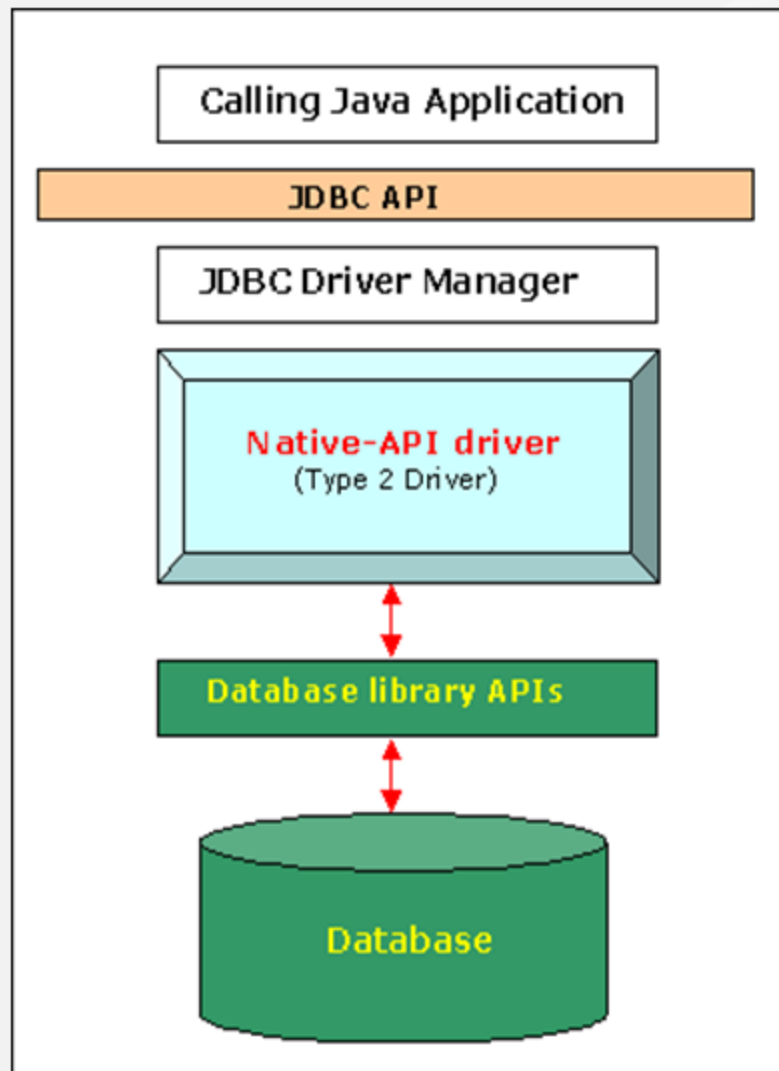
- A JDBC driver is a JDBC API implementation used for connecting to a particular type of database. There are several types of JDBC drivers.
- There are 4 types(generations) of JDBC drives.
- The most commonly used type is type 4, as it has the advantage of being platform-independent. Connecting directly to a database server provides better performance compared to other types.
- The downside of this type of driver is that it's database-specific – given each database has its own specific protocol.

Type 1 : JDBC-ODBC bridge driver



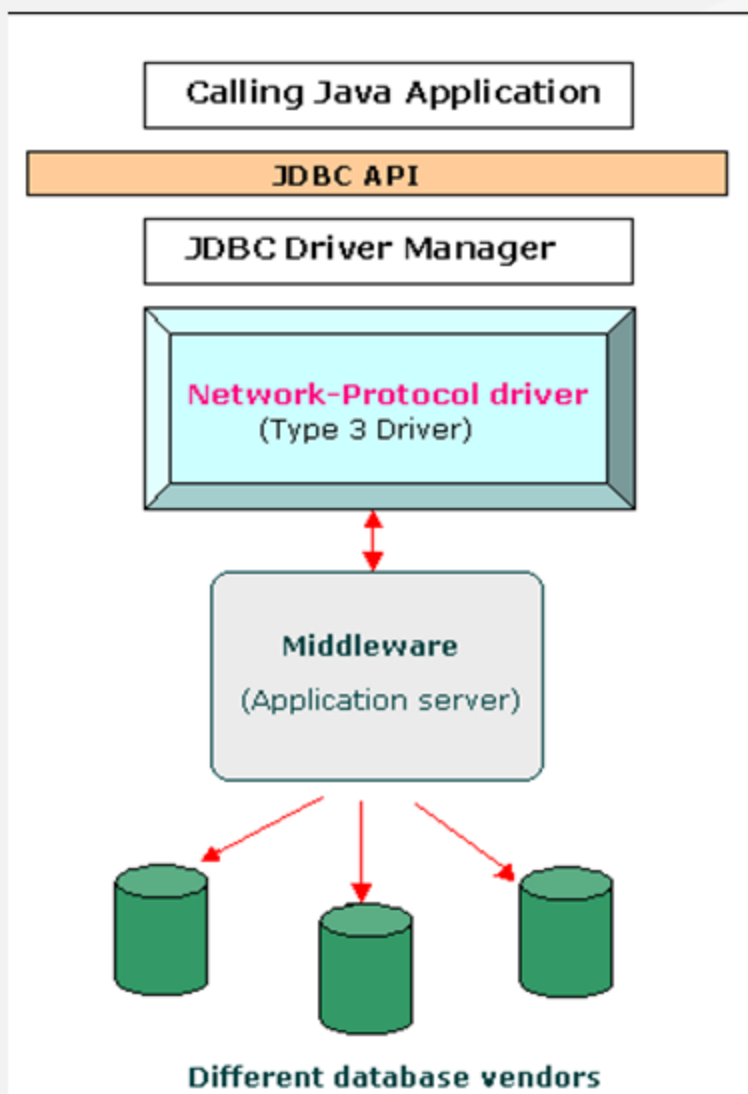
- This driver translates the JDBC calls to ODBC calls. An ODBC bridge then calls ODBC driver of the given database
- This driver is platform-dependent as it makes use of ODBC which in turn depends on native libraries of the underlying operating system.
- Also, use of this driver leads to other installation dependencies; for example, ODBC must be installed on the computer having the driver and the database must support an ODBC driver.

Type 2 : Native API Partly Java Driver



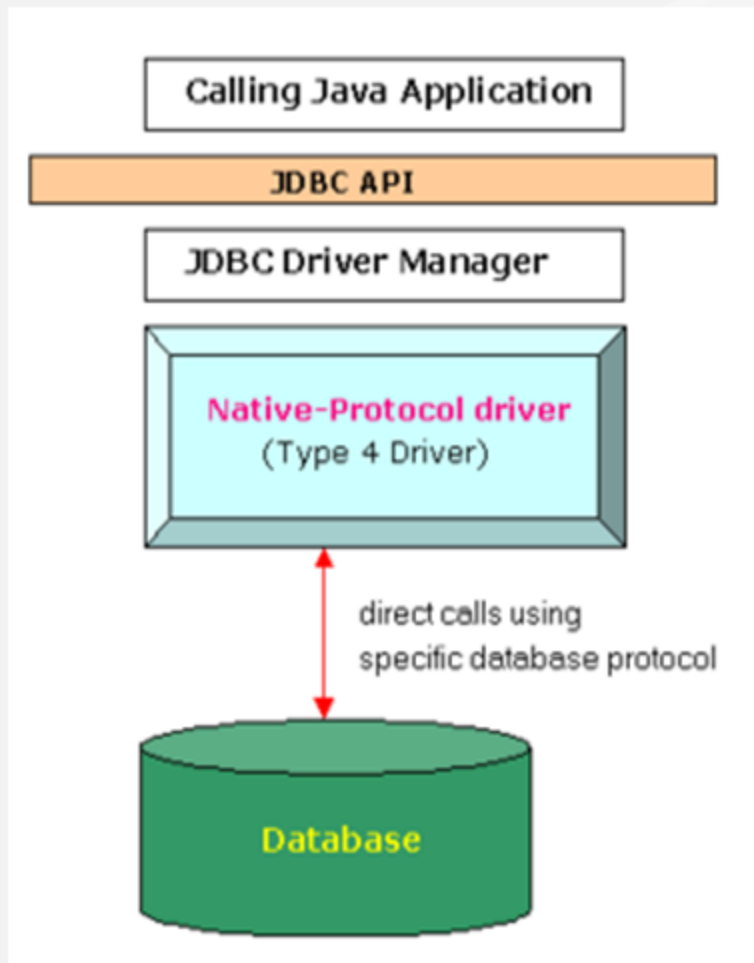
- A type 2 JDBC driver is like a type 1 driver, except the ODBC part is replaced with a native code part
- The native code part is targeted at a specific database product i.e. uses the client-side libraries of the database product. The driver converts JDBC method calls into native calls of the database native API.
- This architecture eliminated the need for the ODBC driver and instead directly called the native client libraries shipped by the database vendors. This was quickly adopted by the DB vendors as it was quick and inexpensive to implement since they could reuse the existing C/ C++ based native libraries.

Type 3 : Pure Java Net Protocol Driver



- A type 3 JDBC driver is purely developed in java
- This driver sends the JDBC interface calls to an intermediate server. The intermediate server then connects to the database on behalf of the JDBC driver.
- The middle-tier (application server) converts JDBC calls directly or indirectly into the vendor-specific database protocol.
- Maintaining middle-tier is a difficult task.

Type 4 : Pure Java Native Driver



- The JDBC type 4 driver, also known as the Direct to Database.
- This driver implementation converts JDBC calls directly into a vendor-specific database protocol. It is implemented for a specific database product.
- Type 4 drivers are thus platform independent.
- They install inside the Java Virtual Machine of the client. This provides better performance than the type 1 and type 2 drivers as it does not have the overhead of conversion of calls into ODBC or database API calls. Unlike the type 3 drivers, it does not need associated software to work

- If you are accessing one type of database, such as Oracle, Sybase, or IBM, the preferred driver type is 4.
- If your Java application is accessing multiple types of databases at the same time, type 3 is the preferred driver.
- Type 2 drivers are useful in situations, where a type 3 or type 4 driver is not available.
- The type 1 driver is not considered a deployment-level driver. It is considered as driver for learning purpose.

Popular JDBC driver names and database URL

RDBMS	JDBC driver name	URL format
MySQL	com.mysql.cj.jdbc.Driver	jdbc:mysql:// hostname/databaseName?serverTimezone=UTC
ORACLE	oracle.jdbc.driver.OracleDriver	jdbc:oracle:thin:@ hostname:portNumber:databaseName

- Registering the driver is the process by which driver class is dynamically loaded into the memory, so it can be utilized as an implementation of the JDBC interfaces.
- Registration should be done only once in your program.
- It can be done in one of two ways.
- The most common approach to register a driver is to use Java's `Class.forName()` method, to dynamically load the driver's class file into memory, which automatically registers it.
- Driver class has a static block which gets executed when it gets loaded into memory. This static block creates instance of the driver and registers it with the Driver Manager.
- This method throws a checked exception **`ClassNotFoundException`**.

```
try
{
    //1.loads the driver class
    //registers the driver with DM
    Class.forName("com.mysql.cj.jdbc.Driver");
}
catch (ClassNotFoundException e)
{
    System.out.println("driver class missing");
    System.exit(1);
}
```

- The second approach is used very rarely to register a driver which makes use of the static `DriverManager.registerDriver()` method.
- This approach is generally used when a non-JDK compliant JVM, such as the one provided by Microsoft is being used.

- A connection to the database can be established by using the `DriverManager.getConnection()` method. Following is the list the three overloaded `DriverManager.getConnection()` methods –
 - ◆ `getConnection(String url)`
 - ◆ `getConnection(String url, Properties prop)`
 - ◆ `getConnection(String url, String user, String password)`
- The most commonly used form of `getConnection()` requires you to pass a database URL, a username, and a password.
- A second form of the `DriverManager.getConnection()` method requires only a database URL. But, in this case, the database URL includes the username and password.
- A third form of the `DriverManager.getConnection()` method requires a database URL and a `Properties` object which holds a set of keyword-value pairs.
- These methods throw a checked exception `SQLException`

Establishing Connection

```
try
{
    //1.loads the driver class
    //registers the driver with DM
    Class.forName("com.mysql.cj.jdbc.Driver");
    String jdbcurl = "jdbc:mysql://localhost:3306/text?serverTimezone=UTC";
    //2. Establishing the connection
    Connection con = DriverManager.getConnection(jdbcurl, "root", "root");
    System.out.println("Connection established");
}
catch (ClassNotFoundException e)
{
    //System.out.println("driver class missing");
    e.printStackTrace();
}
catch (SQLException e)
{
    //System.out.println("Connection failed");
    e.printStackTrace();
}
```

- A Statement is an interface that represents a SQL statement.
- Statement objects get executed, and they generate ResultSet objects in case of select queries, which is a table of data representing a database result set.
- A Connection object is needed to create a Statement object.
- There are three different kinds of statements:
 - ◆ Statement: Used to implement simple SQL statements with no parameters.
 - ◆ PreparedStatement: (Extends Statement.) Used for precompiling SQL statements that might contain input parameters.
 - ◆ CallableStatement: (Extends PreparedStatement.) Used to execute stored procedures

- Statement object can be executed with one of its three execute methods which executes SQL query within the statement
 - ◆ `ResultSet executeQuery (String SQL)` – Returns a `ResultSet` object. This method is used when you expect to get a result set, like for a `SELECT` statement.
 - ◆ `int executeUpdate (String SQL)` – Returns the number of rows affected by the execution of the SQL statement. This method is used to execute SQL statements for which we expect to get a number of rows affected - for example, an `INSERT`, `UPDATE`, or `DELETE` statement.
 - ◆ `boolean execute (String SQL)`: Returns a boolean value of true if a `ResultSet` object can be retrieved; otherwise, it returns false. This method is used to execute SQL DDL statements or when you need to use truly dynamic SQL.
- The data in a `ResultSet` object can be accessed through a cursor. Note that this cursor is not a database cursor. This cursor is a pointer that points to one row of data in the `ResultSet` object. Initially, the cursor is positioned before the first row. You call various methods defined in the `ResultSet` object to move the cursor like `next()`.

Example for simple select query

```
//3. creating the statement
Statement stmt = con.createStatement();
//4. execute the query - providing the query
ResultSet rs = stmt.executeQuery("select empid,salary from emp");
//processing the resultset
while(rs.next())
{
    System.out.print(rs.getInt(1)+" : "); //empid
    System.out.print(rs.getFloat(2)); //salary
    System.out.println();
}
```

- Connection object is used for creating statement. It creates empty container without any query
- Query is passed at the time of executing the statement.
- Resultset is processed by accessing one record at a time using next() to move to next record.
- We can use getX() method to get the data of the columns while iterating through the results where X – is the datatype of the column. We can use either Column Names or Index to get the values using getX() methods.

```
// Inserting data in database
String q1 = "insert into userid values('" +id+ "', '" +pwd
           + "', '" +fullname+ "', '" +email+ "')"
int x = stmt.executeUpdate(q1);
if (x > 0)
    System.out.println("Successfully Inserted");
else
    System.out.println("Insert Failed");
```

- Note : As the columns are varchar type, values are inserted using " around the value.
- executeUpdate() method is used for executing the queries which returns the number of rows affected by the query.

- When a Connection, Statement, or ResultSet object, is no more required, its close() method is called to immediately release the resources it's using.
- Alternatively, use a try-with-resources statement to automatically close Connection, Statement, and ResultSet objects, regardless of whether an SQLException has been thrown. (JDBC throws an SQLException when it encounters an error during an interaction with a data source)
- close() method throws a checked exception SQLException

```
finally
{
    try
    {
        rs.close(); //resultset
        stmt.close(); //statement
        con.close(); //connection
    }
    catch(SQLException e)
    {
        e.printStackTrace();
    }
}
```



JDBC II

What will be covered

- Prepared Statement
- Callable Statement
- Scrollable and updatable result sets
- Managing transactions
- Batch updates
- Transaction isolation levels

- This special type of statement is derived from the more general interface, Statement
- If a Statement object has to be executed many times, it usually reduces execution time to use a PreparedStatement object instead.
- The main feature of a PreparedStatement object is that, unlike a Statement object, it is given a SQL statement when it is created.
- The advantage to this is that in most cases, this SQL statement is sent to the DBMS right away, where it is compiled. As a result, the PreparedStatement object contains not just a SQL statement, but a SQL statement that has been precompiled.
- This means that when the PreparedStatement is executed, the DBMS can just run the PreparedStatement SQL statement without having to compile it first.

- PreparedStatement object provides a feature to execute a parameterized query. We can use the same PreparedStatement and supply with different parameters at the time of execution.
- An important advantage of PreparedStatement is that they prevent SQL injection attacks.
- In short :
 - ◆ Easy to insert parameters into the SQL statement.
 - ◆ Easy to reuse the PreparedStatement with new parameter values.
 - ◆ May increase performance of executed statements.
 - ◆ Enables easier batch updates.
 - ◆ Prevents SQL injection

- To create a PreparedStatement object, connection interface provides the method `prepareStatement(sqlQuery)`
- The parameters in the query are represented by ? which acts as a placeholder for the value.
- Before the prepared statement gets executed, values for those parameters need to be set.
- Creating PreparedStatement object :

```
String sql = "select * from people where id=?";  
  
PreparedStatement preparedStatement =  
    connection.prepareStatement(sql);
```

- Inserting Parameters into a PreparedStatement:

```
preparedStatement.setLong(1, 123);
```

- Inserting parameters at the location of the question mark is done using the setXXX() methods.
- The first argument is the index of the parameter to insert the value for. The second argument is the value to insert into the SQL statement.
- XXX should be replaced by the column's data type
- Executing the PreparedStatement looks like executing a regular Statement. To execute a query, call the executeQuery() or executeUpdate method in case of DML statement

- CallableStatement is used to call stored procedures in a database.
- A stored procedure is like a function or method in a class, except it lives inside the database. Some database heavy operations may benefit performance-wise from being executed inside the same memory space as the database server, as a stored procedure.
- A Callable statement can have output parameters, input parameters, or both.
- The `prepareCall()` method of connection interface will be used to create CallableStatement object.

- Stored procedures are programs that have a database interface. These programs possess the following:
 - ◆ They can have input and output parameters, or parameters that are both input and output.
 - ◆ They can have a return value.
 - ◆ They have the ability to return multiple ResultSets.
- Conceptually in JDBC, a stored procedure call is a single call to the database, but the program associated with the stored procedure may process hundreds of database requests. The stored procedure program may also perform a number of other programmatic tasks not typically done with SQL statements.
- Callable statements can be parameterized like prepared statements. By giving new set of values for execution, callable statement objects can be reused.

- CallableStatement objects may take three types of parameters:
- IN
- IN parameters are handled in the same manner as PreparedStatement. The various set methods of the inherited PreparedStatement class are used to set the parameters.
- OUT
- OUT parameters are handled with the registerOutParameter method. The most common form of registerOutParameter takes an index parameter as the first parameter and an SQL type as the second parameter. This tells the JDBC driver what to expect for data from the parameter when the statement is processed.
- INOUT
- INOUT parameters require that the work for both IN parameters and OUT parameters be done. For each INOUT parameter, you must call a set method and the registerOutParameter method before the statement can be processed. Failing to set or register any parameter results in an SQLException being thrown when the statement is processed.

- Example of store procedure in mysql with in and out parameters

```
DELIMITER $$

CREATE PROCEDURE GetOrderCountByStatus (
    IN  orderStatus VARCHAR(25),
    OUT total INT
)
BEGIN
    SELECT COUNT(orderNumber)
    INTO total
    FROM orders
    WHERE status = orderStatus;
END$$

DELIMITER ;
```

```
cs = con.prepareStatement("{call getSalary(?,?)}");  
cs.setInt(1, eid); //in parameter  
cs.registerOutParameter(2, Types.FLOAT); //out parameter  
//4 Executing callable statement  
cs.execute();  
float sal = cs.getFloat(2);  
System.out.println("Salary of "+eid+" is "+sal);
```

- For processing in parameters setXXX method is used just like a prepared statement
- For processing out parameters, registerOutParameter() method is used.
- For executing callable statement, any method of execution can be used.
- Out parameters can be retrieved after the execution by using getXXX() method by passing out parameter index.

- Whenever we create an object of ResultSet by default, it allows us to retrieve in the forward direction only and we cannot perform any modifications on ResultSet object. Therefore, by default, the ResultSet object is non-scrollable and non-updatable ResultSet.
- Scrollable means that once the ResultSet object has been created, we can traverse through fetched records in any direction, forward and backward, as we like. This provides the ability to read the last record, first record, next record, and the previous record.
- The type of ResultSet object we like to create must be explicitly declared to the Statement object through defined scroll type constants.
 - ◆ `ResultSet.TYPE_FORWARD_ONLY`: This is the default type.
 - ◆ `ResultSet.TYPE_SCROLL_INSENSITIVE`: Enables back and forth movement, but is insensitive to ResultSet updates.
 - ◆ `ResultSet.TYPE_SCROLL_SENSITIVE`: Enables back and forth movement, but is sensitive to ResultSet updates.

- Creating an updatable ResultSet means that the record it points to is not only be traversable but also be updatable. The changes will immediately be persisted in the database and reflected by the ResultSet object in real time.
- Creating updatable result set should be mentioned at the time of creating statement object
- Following are the constants used for creating updatable result sets
 - ◆ CONCUR_READ_ONLY, which means that the ResultSet is not updatable.
 - ◆ CONCUR_UPDATABLE, which signifies the opposite, meaning the ResultSet is updatable.

```
//3 creating the statement
Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
    ResultSet.CONCUR_UPDATABLE);

//4 executing the statement
//scrollable and updatable
ResultSet rs = stmt.executeQuery("select * from product");
```

- All types of statement provides overloaded methods for creation which accepts result set type and result set concurrency as arguments.
- Once the result set becomes scrollable, it provides additional methods for scrolling other than next() like
 - ◆ previous()
 - ◆ absolute()
 - ◆ relative()
 - ◆ beforeFirst()
 - ◆ afterLast()

```
rs.first();  
System.out.println("FIRST RECORD...");  
System.out.println(rs.getInt(1) + " -> " + rs.getString(2));  
rs.absolute(3);  
System.out.println("THIRD RECORD...");  
System.out.println(rs.getInt(1) + " -> " + rs.getString(2));  
rs.last();
```

- Update all columns of the database or provide the values to all columns of the database by using the following generalized method which is present in the ResultSet interface.

rs.updateXXX(int colno, XXX val);

- Record inserted, updated or deleted from result set can be permanently changed into the database by using methods in ResultSet interface

insertRow(), updateRow() or deleteRow()

```
Statement st = con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,  
    ResultSet.CONCUR_UPDATABLE);  
ResultSet rs = st.executeQuery("select * from person");  
rs.next();  
rs.updateInt(1, 1001);  
rs.updateRow();  
System.out.println("1 ROW UPDATED...");
```

- There are times when you do not want one statement to take effect unless another one completes.
- A transaction represents a group of operations, used to perform a task.(All successful or none successful)
- A transaction is a set of commands, it will take our database from one consistent state to another consistent state.
- Example : At the time of registration of the user, the registration information may be split and inserted into different tables. When insertion into all different tables is successful, then only it should be considered as successful. When insertion fails in any table, it should be considered as failure and no insertion should be committed.

- We can get the Transaction support in JDBC from Connection interface. The Connection interface given 3 methods to perform Transaction Management in JDBC.
 - ◆ `setAutoCommit()`
 - ◆ `commit()`
 - ◆ `rollback()`
- When a connection is created, it is in auto-commit mode. This means that each individual SQL statement is treated as a transaction and is automatically committed right after it is executed.
- The way to allow two or more statements to be grouped into a transaction is to disable the auto-commit mode. This is demonstrated in the following code, where con is an active connection:

```
con.setAutoCommit(false);
```
- After the auto-commit mode is disabled, no SQL statements are committed until you call the method `commit` explicitly. All statements executed after the previous call to the method `commit` are included in the current transaction and committed together as a unit.

```
con.setAutoCommit(false);
```

```
Statement stmt=con.createStatement();
```

```
stmt.executeUpdate("insert into user420 values(190,'abhi',40000)");
```

```
stmt.executeUpdate("insert into user420 values(191,'umesh',50000)");
```

```
con.commit();
```

- Whenever the commit method is called (either automatically when auto-commit mode is enabled or explicitly when it is disabled), all changes resulting from statements in the transaction are made permanent.
- The statement `con.setAutoCommit(true);` enables auto-commit mode, which means that each statement is once again committed automatically when it is completed

- With batch updates, instead of updating rows of a table one at a time, you can direct JDBC to execute a group of updates at the same time. Statements that can be included in the same batch of updates are known as batchable statements.
- Executing a SELECT statement in a batch, will result into a `BatchUpdateException`.
- `addBatch()` method from all types of statements is used for adding a statement in the batch.
- `executeBatch()` method is used for executing a batch of statements which returns `int[]` type of value indicating no of rows affected by each statement.
- If any statement fails, it results into `BatchUpdateException`.
- Explicitly commit when all the statements in a batch gets successfully executed

```
PreparedStatement preps = conn.prepareStatement(
    "UPDATE DEPT SET MGRNO=? WHERE DEPTNO=?");
ps.setString(1,mgrnum1);
ps.setString(2,deptnum1);
ps.addBatch();

ps.setString(1,mgrnum2);
ps.setString(2,deptnum2);
ps.addBatch();
int [] numUpdates=ps.executeBatch();
for (int i=0; i < numUpdates.length; i++) {
    if (numUpdates[i] == SUCCESS_NO_INFO)
        System.out.println("Execution " + i +
            ": unknown number of rows updated");
    else
        System.out.println("Execution " + i +
            "successful: " numUpdates[i] + " rows updated");
}
conn.commit();
```

- Transactions specify an isolation level that defines how one transaction is isolated from other transactions.
- Isolation is the separation of resource or data modifications made by different transactions.
- Isolation levels are described for which concurrency side effects are allowed, such as dirty reads or phantom reads.
- Dirty reads occur when:
 - ◆ Transaction A inserts a row into a table.
 - ◆ Transaction B reads the new row.
 - ◆ Transaction A rolls back.
- Phantom reads occur when:
 - ◆ Transaction A reads all rows that satisfy a WHERE clause on an SQL query.
 - ◆ Transaction B inserts an additional row that satisfies the WHERE clause.
 - ◆ Transaction A re-evaluates the WHERE condition and picks up the additional row.

- There are five levels of transaction isolation listed from least to most restrictive, they are as follows:
- `JDBC_TRANSACTION_NONE`
- This is a special constant indicating that the JDBC driver does not support transactions.
- `JDBC_TRANSACTION_READ_UNCOMMITTED`
- This level allows transactions to see uncommitted changes to the data. All database anomalies are possible at this level.
- `JDBC_TRANSACTION_READ_COMMITTED`
- This level means that any changes made inside a transaction are not visible outside it until the transaction is committed. This prevents dirty reads from being possible.
- `JDBC_TRANSACTION_REPEATABLE_READ`
- This level means that rows that are read retain locks so that another transaction cannot change them when the transaction is not completed. This disallows dirty reads and nonrepeatable reads. Phantom read are still possible.
- `JDBC_TRANSACTION_SERIALIZABLE`
- Tables are locked for the transaction so that WHERE conditions cannot be changed by other transactions that add values to or remove values from a table. This prevents all types of database anomalies.

- Integer Value for Isolation level
 - ◆ TRANSACTION_NONE = 0
 - ◆ TRANSACTION_READ_COMMITTED = 2
 - ◆ TRANSACTION_READ_UNCOMMITTED = 1
 - ◆ TRANSACTION_REPEATABLE_READ = 4
 - ◆ TRANSACTION_SERIALIZABLE = 8
- You can get the current isolation level by using method `getTransactionIsolation()`, and also set the isolation by `setTransactionIsolation(int isolationlevelconstant)` methods respectively.

```
// Support for Transaction Isolation Level (Connection)
System.out.println("Transaction Isolation level= "
    + con.getTransactionIsolation());
// Setting Transaction Isolation Level
// You can set Its String Value or its int value
con.setTransactionIsolation(2);
```